Open Object Formats for Near Data Processing (and more!)

Daniel Bittman

CTO, Elephance Memory



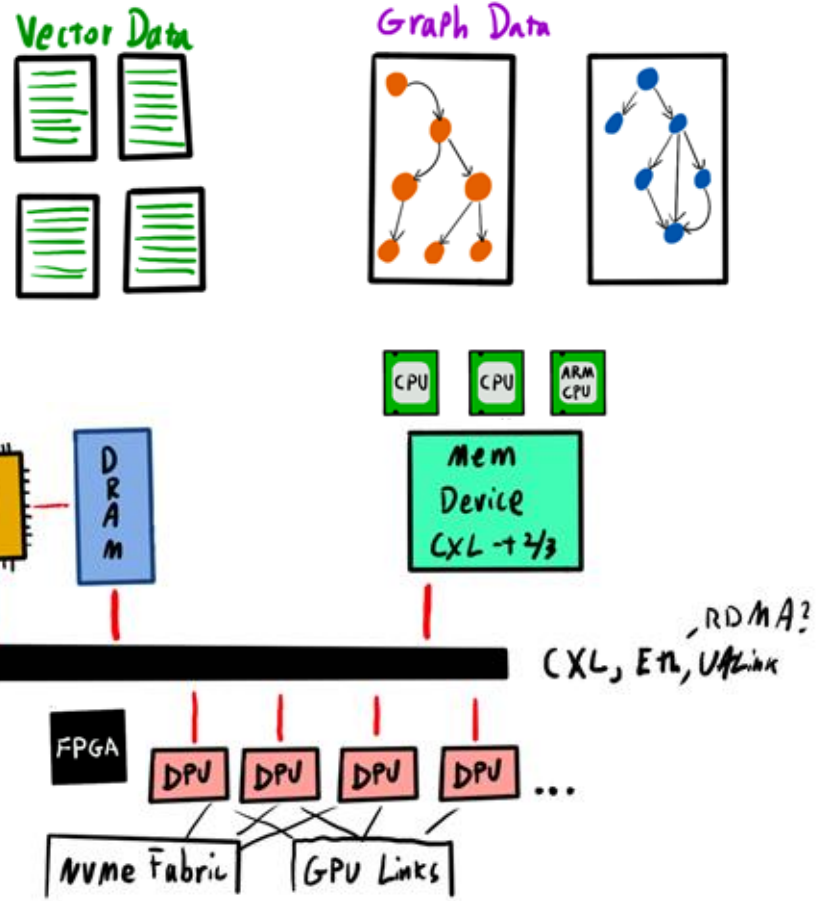
OCP FUTURE TECHNOLOGIES SYMPOSIUM

October 14, 2026
San Jose, California

Application demands increasing

AI, HPC, Graph Processing

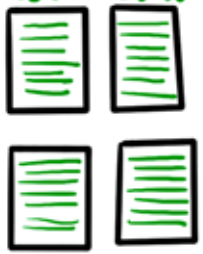
→ Throughput
→ Parallelism
→ Capacity



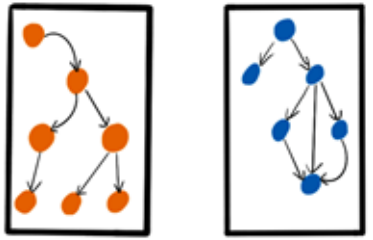
Data Gravity → NDP

graphs are challenging

Vector Data



Graph Data



Serialization + movement overheads kill performance

Data

Compute

Interconnects

Devices

Programming Model Limitations

System

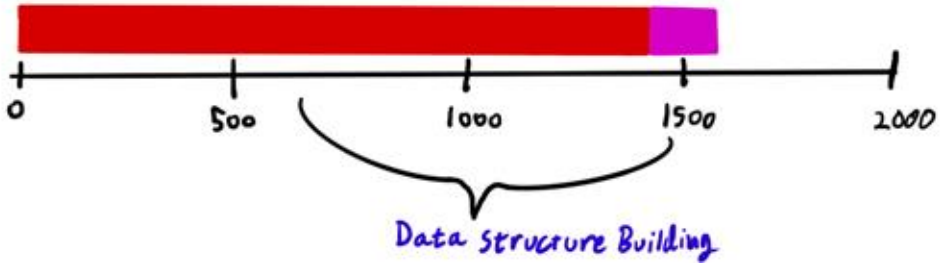
- Page fault storms
- TLB coherence
- Page Table mgmt

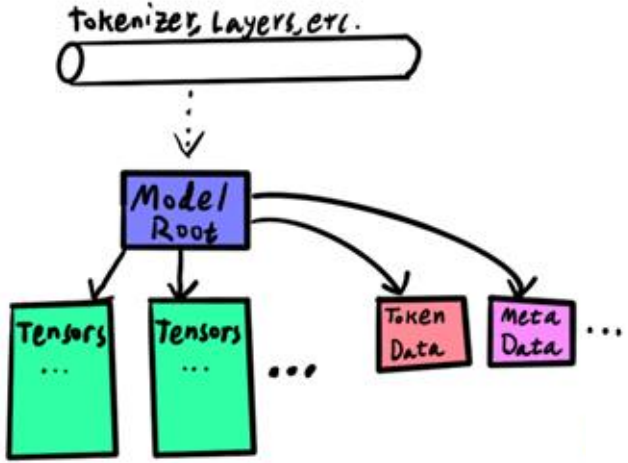
App

- Serialization
- Runtime Data
- Limited Locality ctrl

Model Load

→fewer Page faults ($\sim 10^5 \rightarrow 10^2$)
→all model data computable
→recomputable data stored (optional)





Stable in memory formats

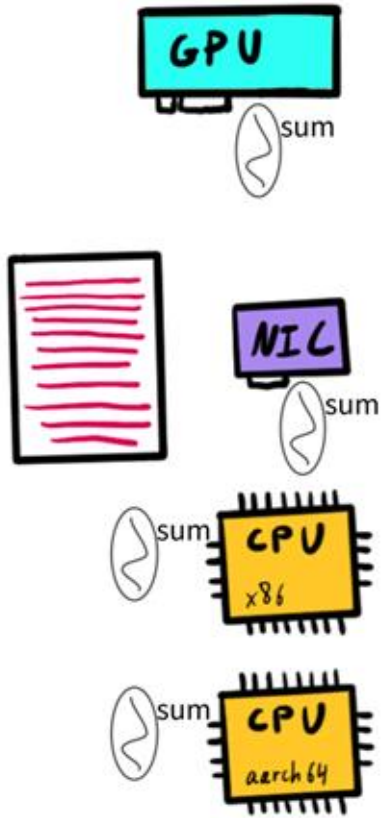
+

Global compute metaphors

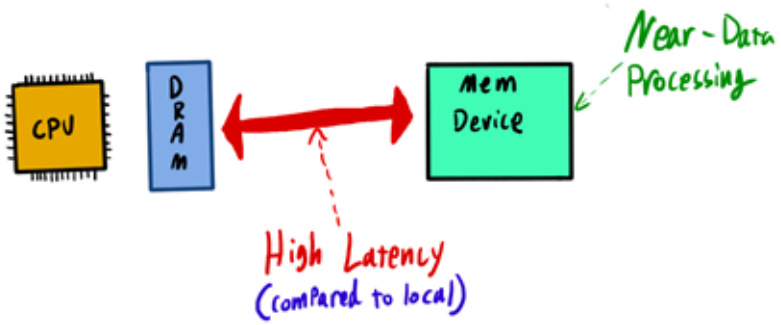
Independent Scaling of Memory+Compute

NDP, PIM, full stack Parallelism

Wide, zero-copy sharing of directly computable data

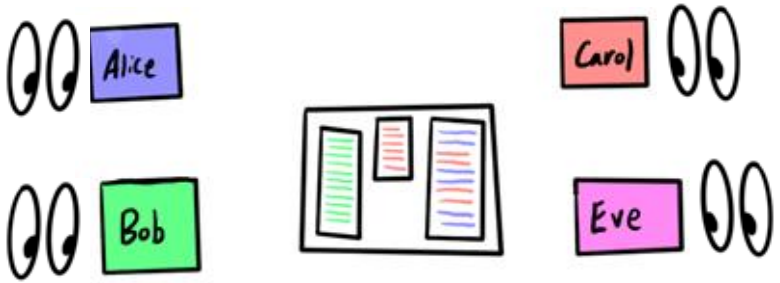


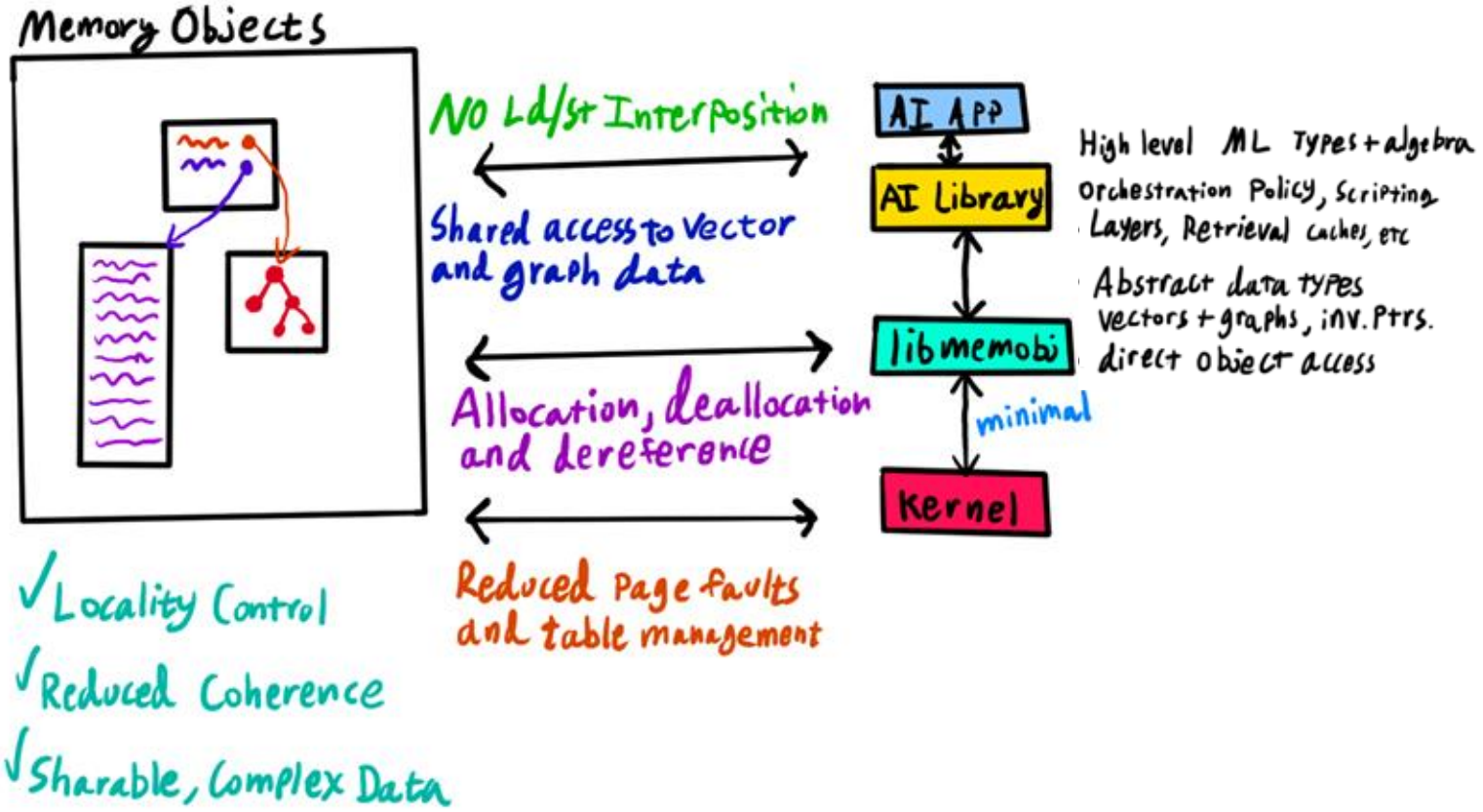
Support Memory intensive applications



Data-centric Programming model
→ Direct access to data
→ Low coordination sharing
Even for complex data structures

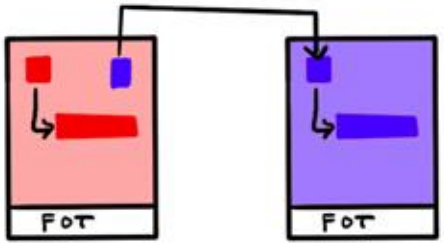
Domain specific libraries provide memory-object-based types+structures (eg HNSW)




Memory Objects
NO Ld/st Interposition
Shared access to vector and graph data
Allocation, deallocation and dereference
Reduced Page faults and table management
AI App
AI Library
libmemobj
Kernel
minimal
High level ML Types + algebra
Orchestration Policy, Scripting Layers, Retrieval caches, etc
Abstract data types vectors + graphs, inv. Ptrs. direct object access
✓Locality Control
✓Reduced Coherence
✓Sharable, Complex Data

Self-Contained → Fluid Movement

→ Invariant of Perspective




```rust
struct GraphNode {
    // ...
    vector: ArrowBuffer<UInt32Type>,
}
fn arrow_sum(object: Object<GraphNode>) -> Option<u32> {
    let buf = object.base().vector.buffer();
    let sc = arrow::buffer::ScalarBuffer::<u32>::new(buf, 0, LEN);
    let prim_array = arrow::array::PrimitiveArray::<UInt32Type>::new(sc, None);
    arrow::compute::sum(&prim_array)
}
```

Low overhead

Aggressive caching

*node → r = Inviar::resolve(node)

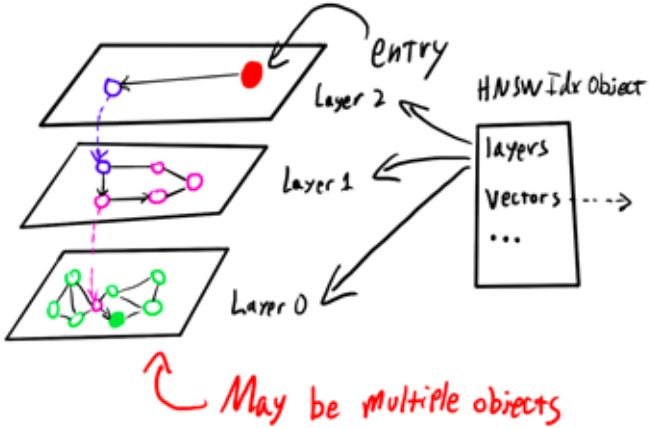
*r

Cached result

Pointer Compression

where do objects come from?

- → Created explicitly
- → As part of a data structure
- → Via allocation (with hints!)



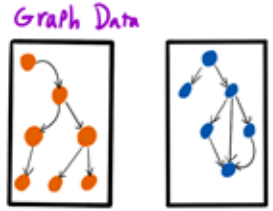
App- or data structure-specific semantics inform placement without removing low-level control

→ Locality Control

Key Value Store : 3.8 – 5.3x throughput

Also important for page faults!





→ Memory Semantics

Reduce kernel involvement

Control Coherence + Mapping

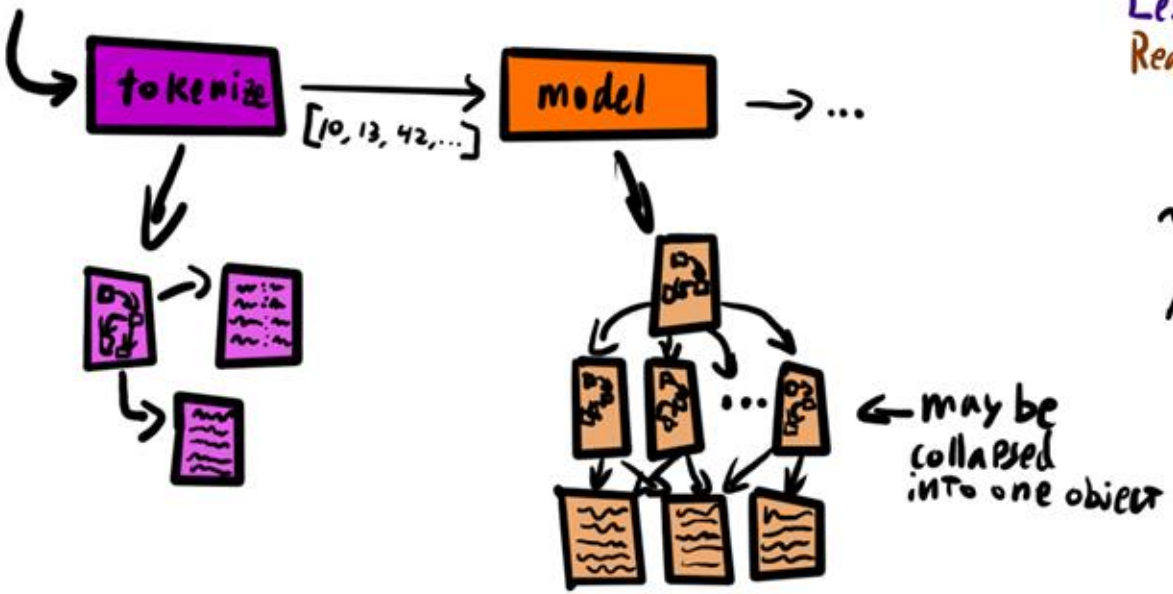
→ Interoperability

Direct Unix Support

Stable in-memory + open formats

Arrow, Arrow Compute,
pNFS, FamFS, substrait
Refine, Rust, C++
CXL, TPP, NDP, folios
Linux, Twizzler

"I got it one piece at a time..."



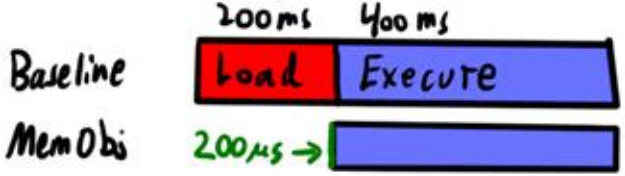
Fewer Page faults

Less Address Space Management

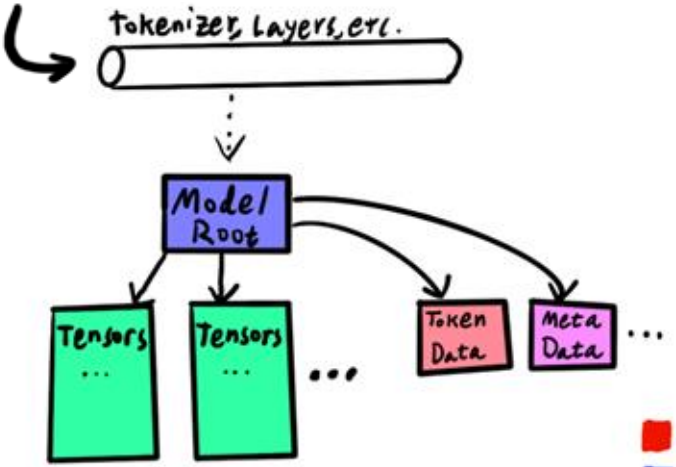
Reduced copying / wasted work

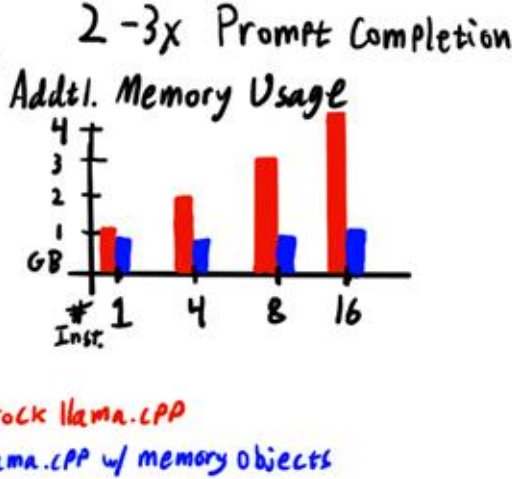
~10,000 faults → 80

No impact on runtime



"I got it one piece at a time..."



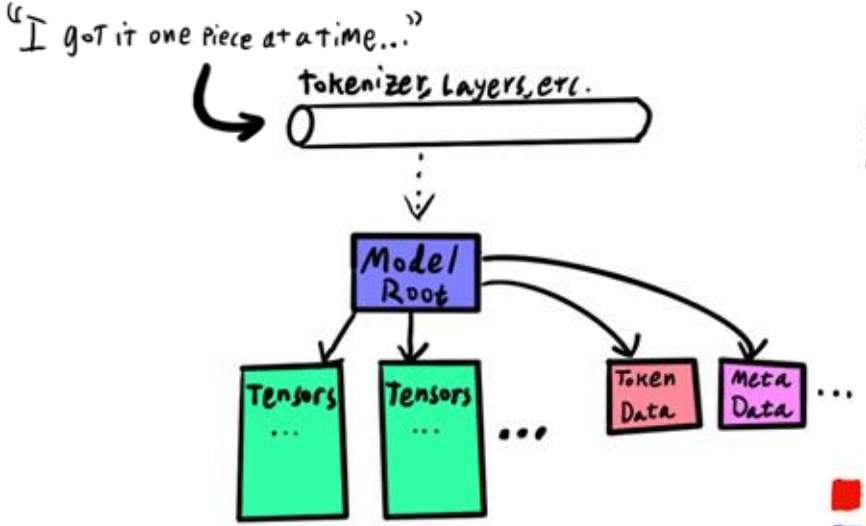


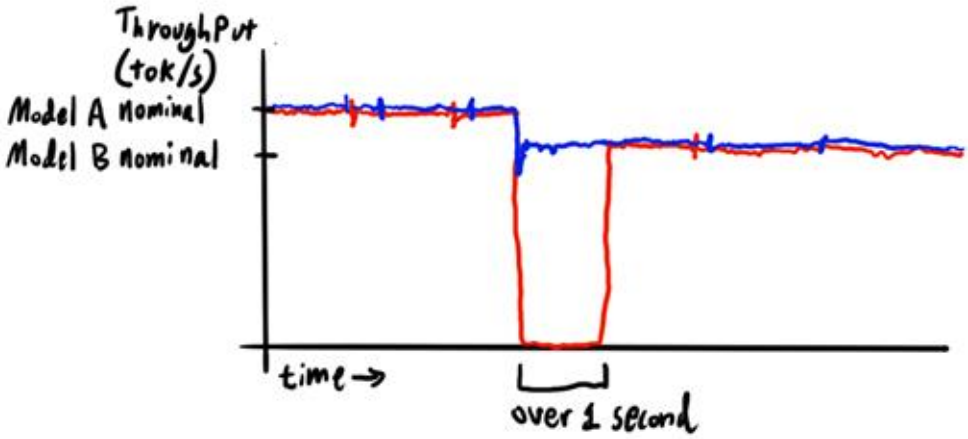
✓Decode performance slightly improved (expected)

✓Model load and switch time vastly improved



Page Fault Reduction: ~$10^5 \rightarrow 10^2$





Stock llama.cpp

llama.cpp w/ memory objects

✓Decode performance slightly improved (expected)

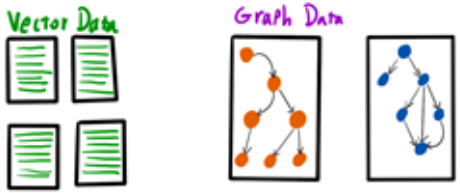
✓Model load and switch time vastly improved



Open Formats: A call to Action

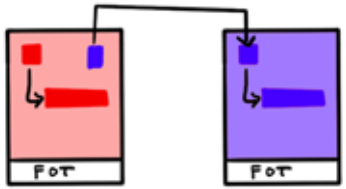
- → Work With Industry Partners
  - → Standardize on open formats for Data + Compute
  - → Algebras and Sharable Data
- → Work With Hardware Partners
  - → Prove out benefits
  - → Solve Software scaling problems for sharing
- → Work With OCP + Community
  - → Updating benchmarks w/ key metrics
  - → Publishing standardization + Community interop

Both Applications and emerging hardware demand new programming models for in-memory data



Data-centric Programming model
- → Direct access to data
- → Low coordination sharing

Even for complex data structures
- ✓ Parallelism, large and small
- ✓ Developer-friendly flexibility
- ✓ wide sharing of large, complex data



Thank You!

Looking for Partners!

Questions?

Daniel Bittman
daniel bittman@elephance memory. com

Pankaj Mehra
pankaj.mehra@ elephance memory.com

elephance memory.com



OCP FUTURE TECHNOLOGIES SYMPOSIUM

October 14, 2026
San Jose, California

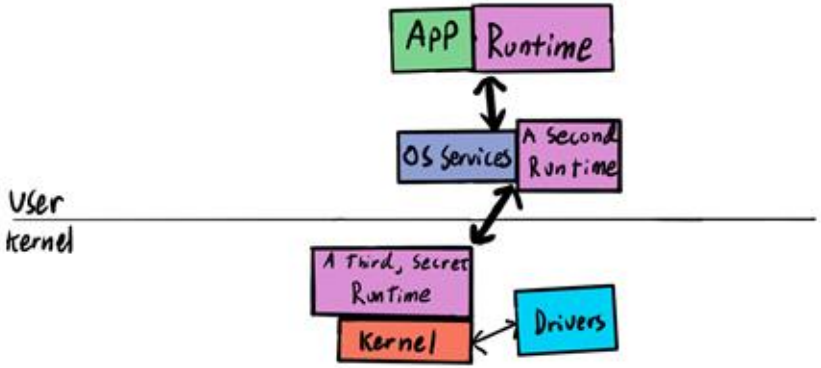


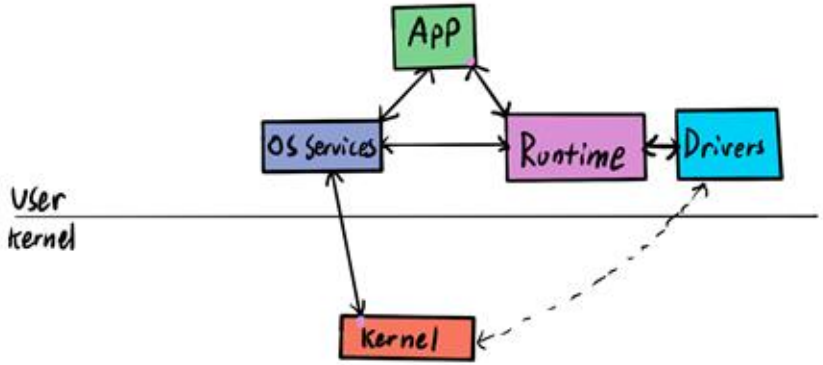
[Diagrams/Charts]

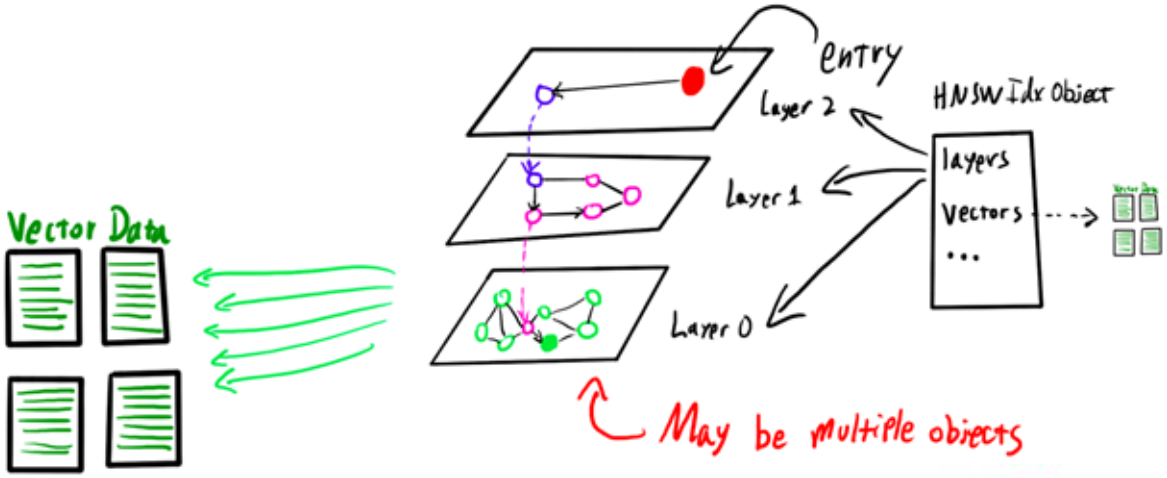


Backup slides




App
Runtime
OS Services
A Second Runtime
User
Kernel
A Third, Secret Runtime
Kernel
Drivers


App
OS Services
Runtime
Drivers
User
Kernel
Kernel



	Load
HNSW (File)	27 ± 3 ms
HNSW (Memos)	0.2 ± 0.1 ms
Vector DB	7 ± 4 ms

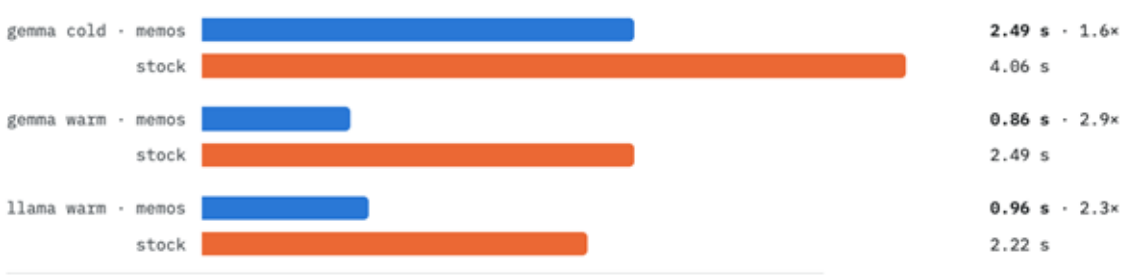
virtual addresses

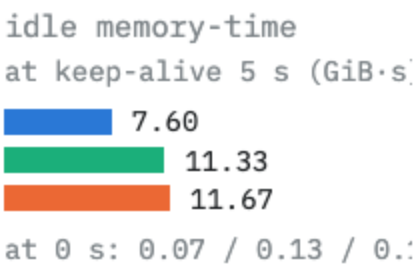
more flexible

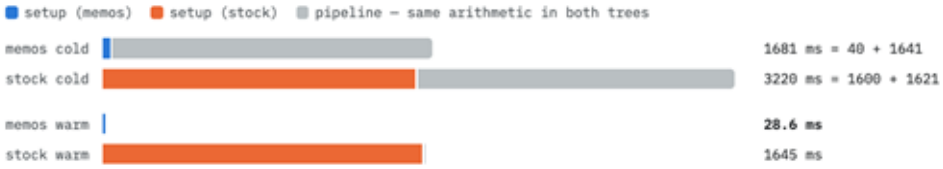
HNSW	baseline	1.0 ± 0.02
	invariant	1.01 ± 0.02
	w/ Arrow	0.78 ± 0.01
k-means	baseline	1.0 ± 0.1
	invariant	1.0 ± 0.1

(normalized to baseline)

FAISS: 3% - 9% overhead

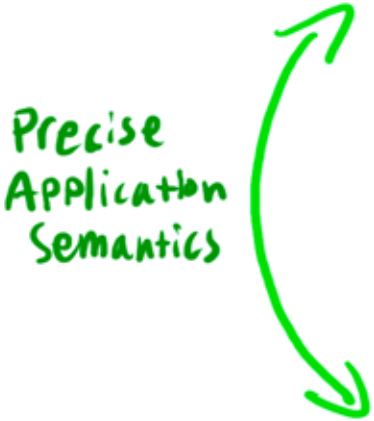






N	TRUE TOTAL (ΣPSS)	
	MEMOS+TWZM	STOCK
1	1.24 GB	1.27 GB
4	**1.12 GB**	2.00 GB
8	**1.21 GB**	2.97 GB
16	**1.37 GB**	**4.91 GB**

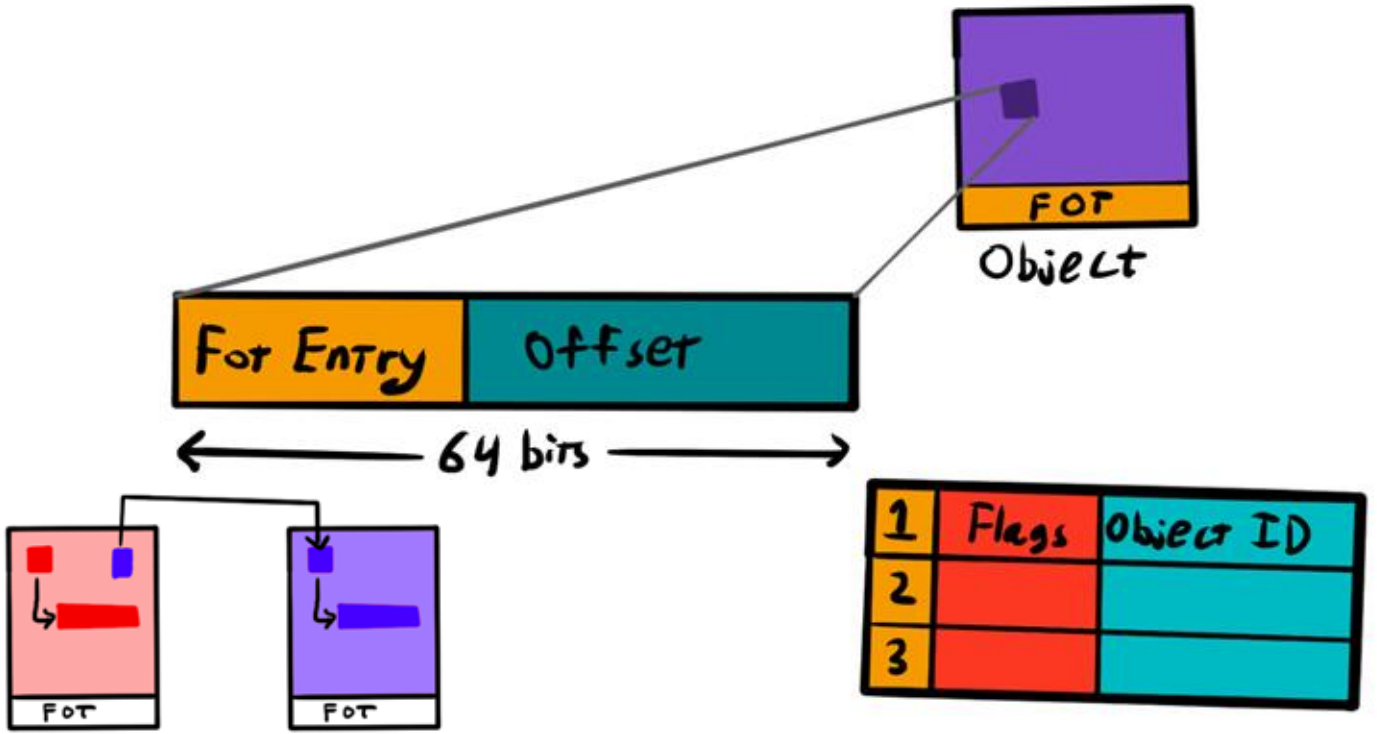
Depending on appending



Immutability changes everything

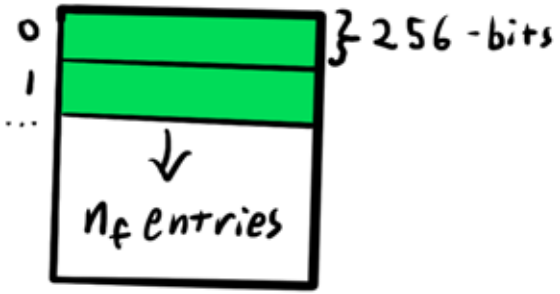


Safe, fast APIs for shared memory

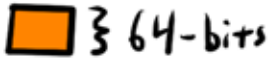

FOT
Object
For Entry
Offset
64 bits
FOT
FOT
1
Flags
Object ID
2
3

Twizzler

FOT

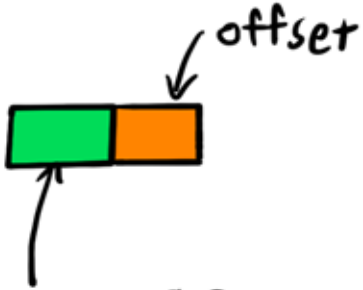


Inv. Pointers



n_p Pointers

Fat-Pointer



n_p fat-pointers

Sparse

Twizzler ~2.3x

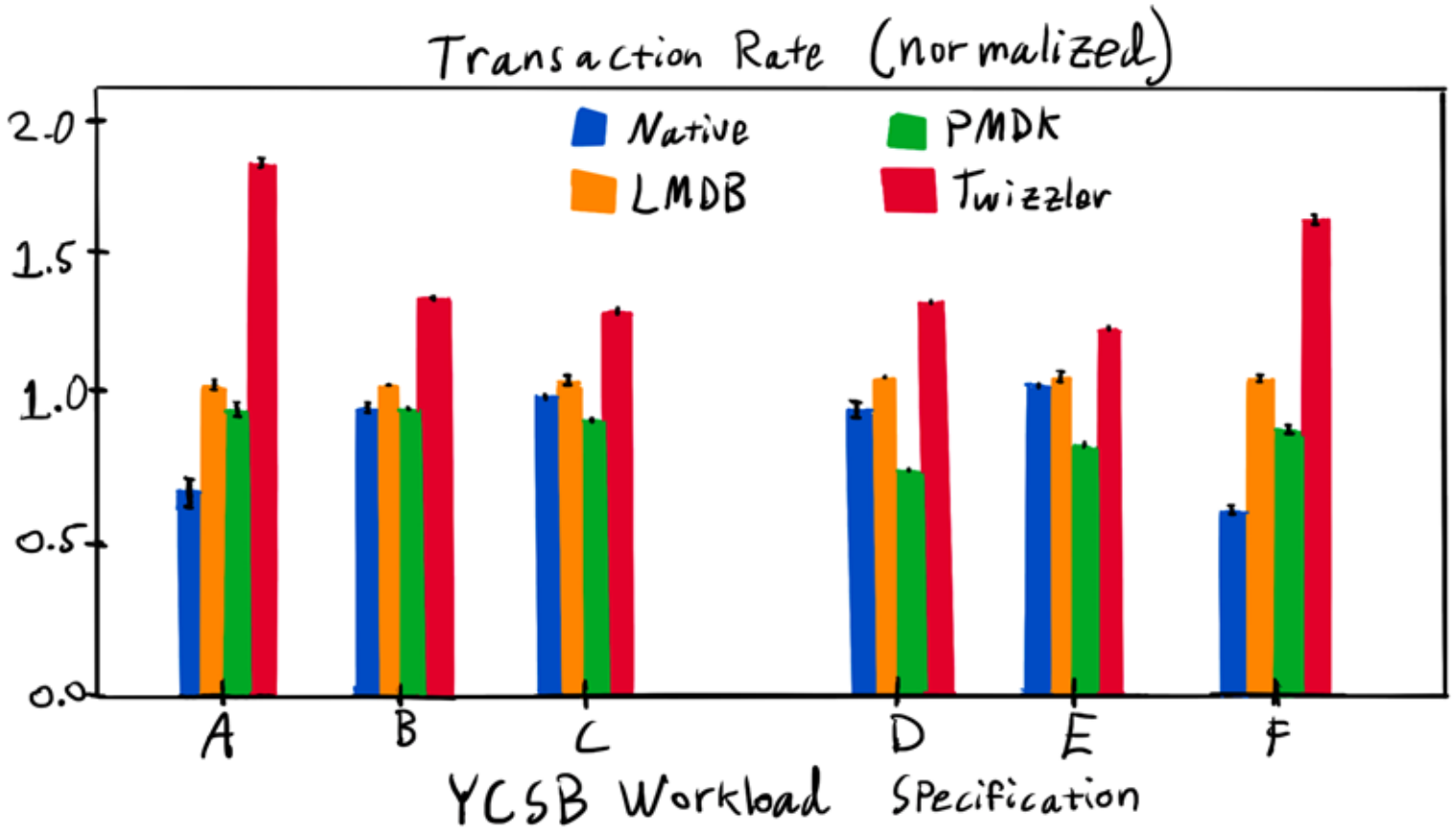
Fat-Pointer ~ 3x

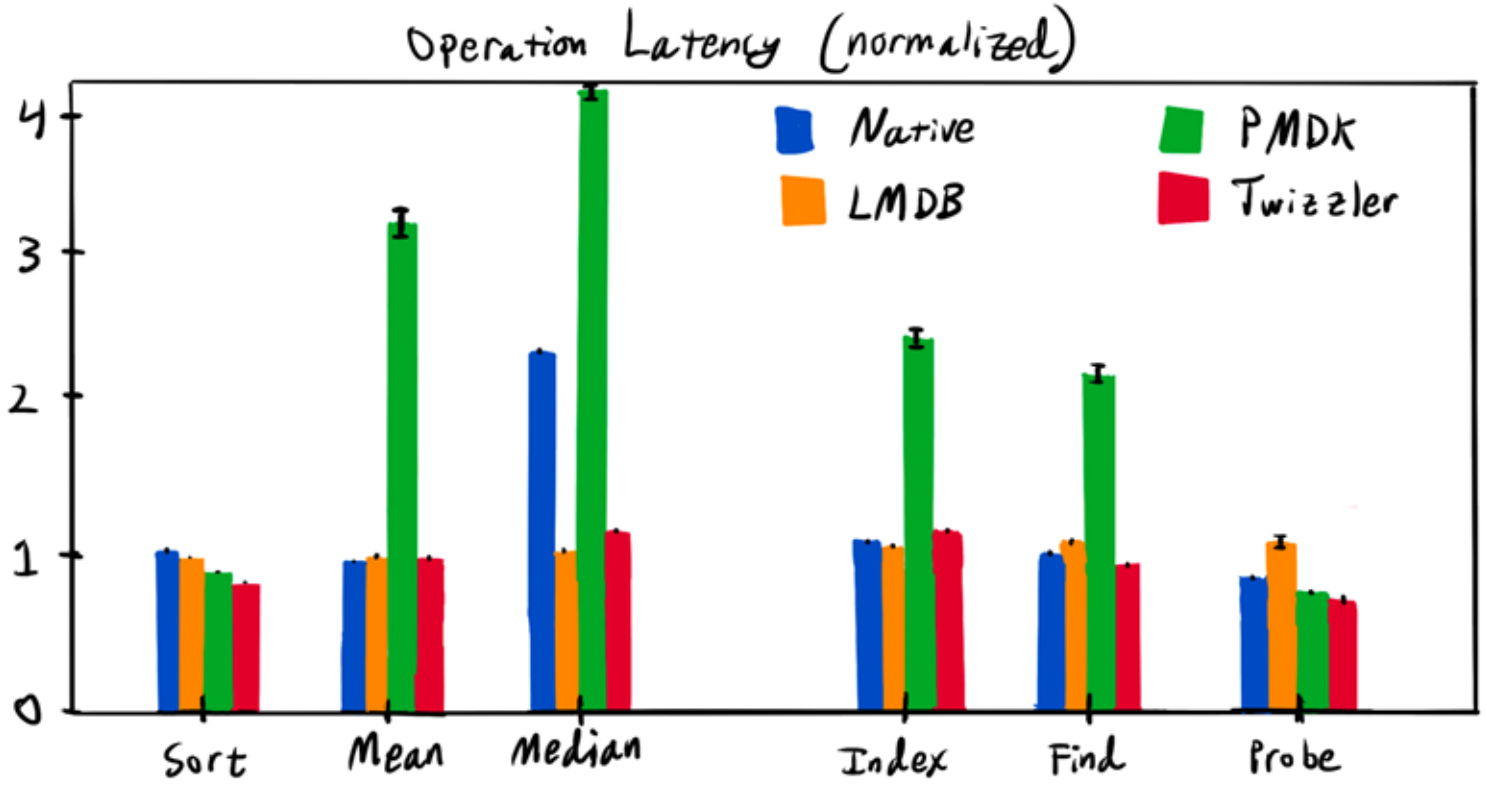
Dense

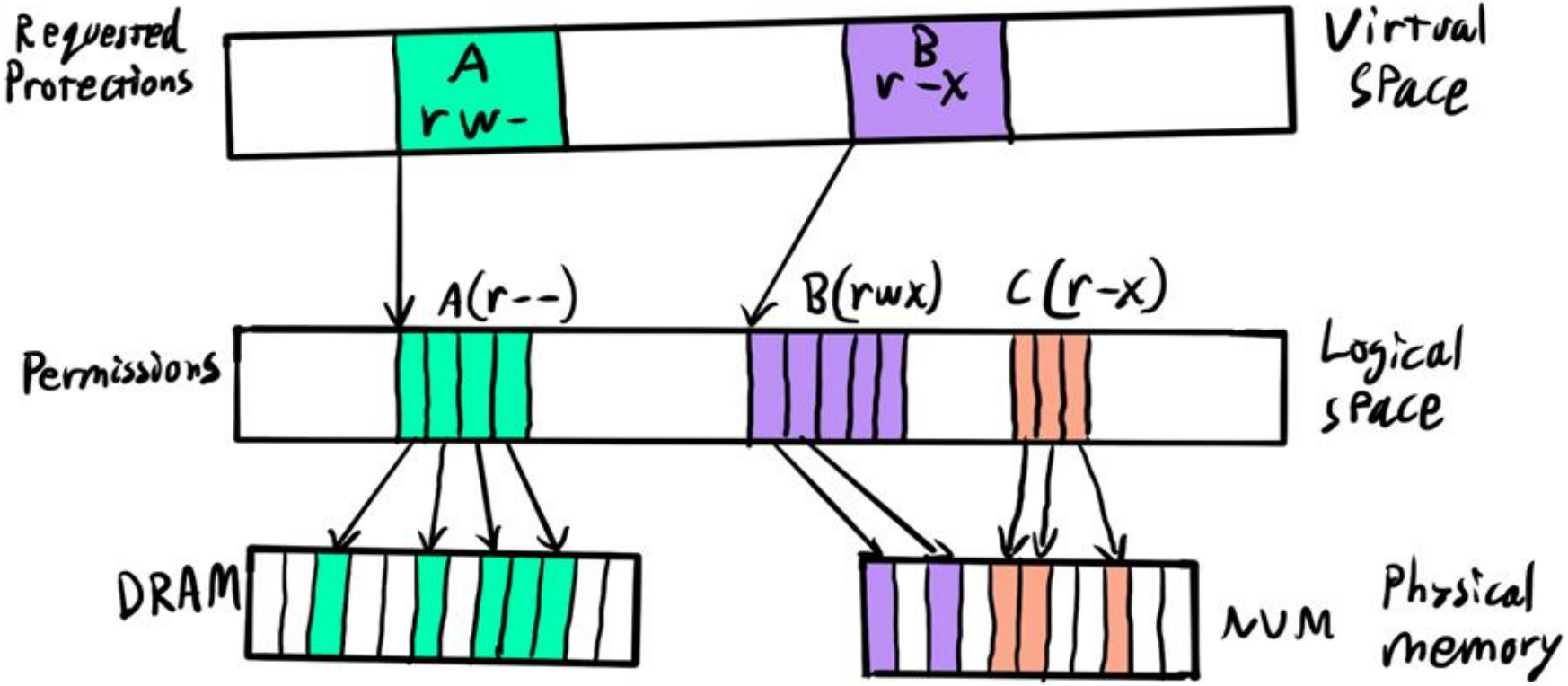
twizzler ~ $(1+\varepsilon)x$

Fat-Pointer ~ 3x

Intra-object	0.4 ± 0.1 ns
Inter-object (uncached)	27.9 ± 0.1 ns
Inter-object (cached)	3.2 ± 0.1 ns


Transaction Rate (normalized)
Native
PMDK
LMDB
Twizzler
2.0
1.5
1.0
0.5
0.0
A
B
C
D
E
F
YCSB Workload Specification


Operation Latency (normalized)
Native
PMDK
LMDB
Twizzler
4
3
2
1
0
Sort
Mean
Median
Index
Find
Probe


Requested Protections
A
rw-
B
r-x
Virtual Space
A(r--)
B(rwx)
C(r-x)
Permissions
Logical space
DRAM
NUM
Physical memory